REGISTERS AND REGISTER FILES

Registers and Register Files

- A **register file** is an array of processor registers in a central processing unit (CPU)
- The Register File is the highest level of the memory hierarchy.
- Register file can be implemented using SRAM with multiple ports.
- Such RAMs are distinguished by having dedicated read and write ports, whereas ordinary SRAMs will usually read and write through the same ports.
- Two roles
  - User-visible registers
  - Control and status registers

User-Visible Registers

- General Purpose
- Data
- Address
- Condition Codes

General Purpose Registers

- True general purpose registers – register can contain the operand for any Opcode
- Restricted – used for specific operations – floating point and stack operations. (dedicated registers)
- Data registers – used only to hold data and cannot be employed in the calculation of an operand address – Accumulator (AC)
- Address registers
  - Segment registers – holds the address of the base of the segment.
  - Index registers – used for indexed addressing and may be auto-indexed
  - Stack pointer – points to the top of the stack (if there is a user-visible stack addressing, stack is in memory)

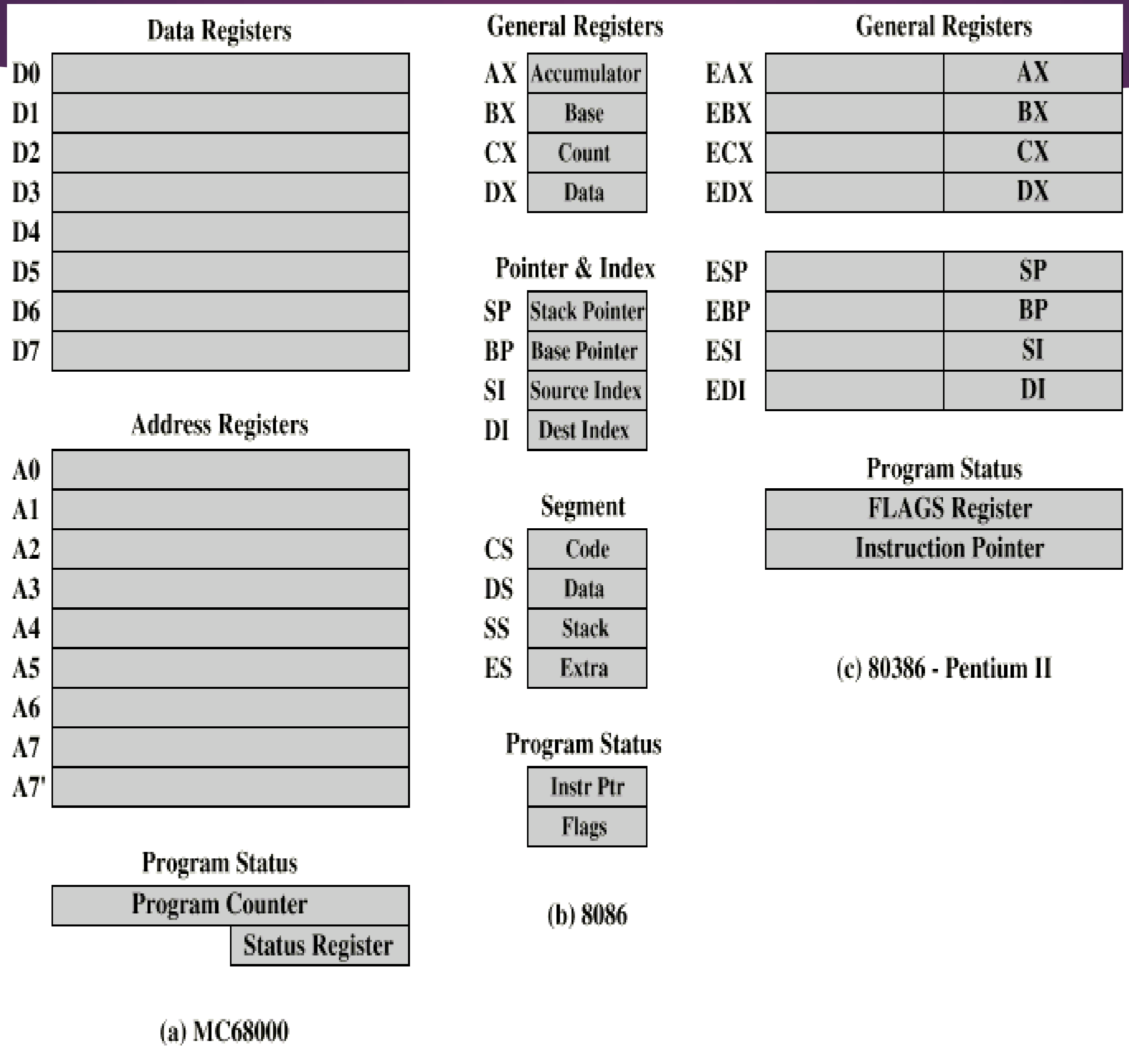
Condition Code Registers – Flags

- Condition codes are bits set by the CPU hardware as the result of operations.
- Machine instructions allow these bits to be read by implicit reference.
- Programmer cannot alter them
- In some machines, sub-routine call will result in the automatic saving of all user-visible registers, to be restored on return.
- Sets of individual bits, flags
  - e.g. result of last operation was zero
- Can be read by programs
  - e.g. Jump if zero – simplifies branch taking

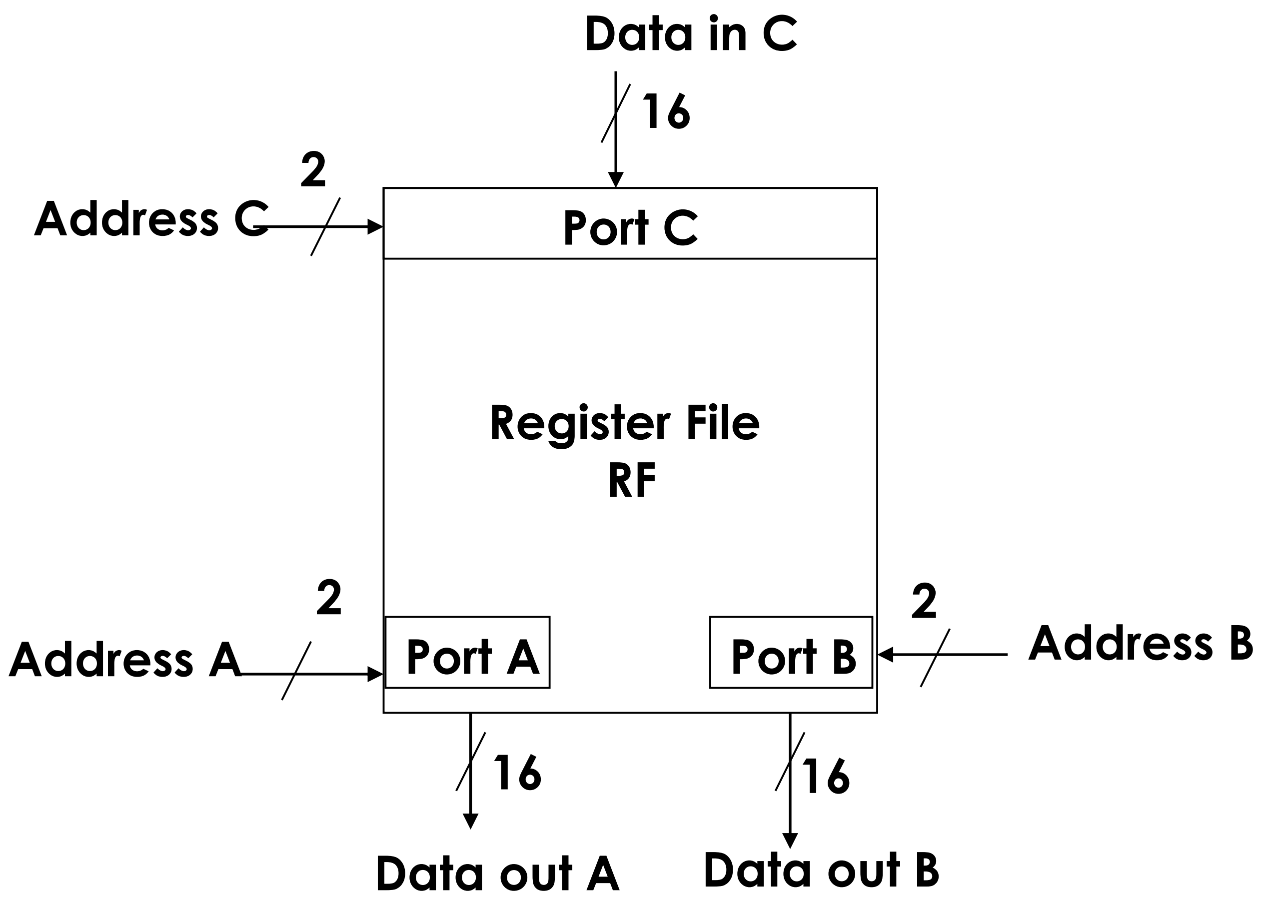
Control & Status Registers

- Not visible to the user
- May be visible in a control or operating system mode (supervisory mode)
- Registers essential to instruction execution:
  - Program Counter (PC)
  - Instruction Register (IR)
  - Memory Address Register (MAR) – connects to address bus
  - Memory Buffer Register (MBR) – connects to data bus, feeds other registers

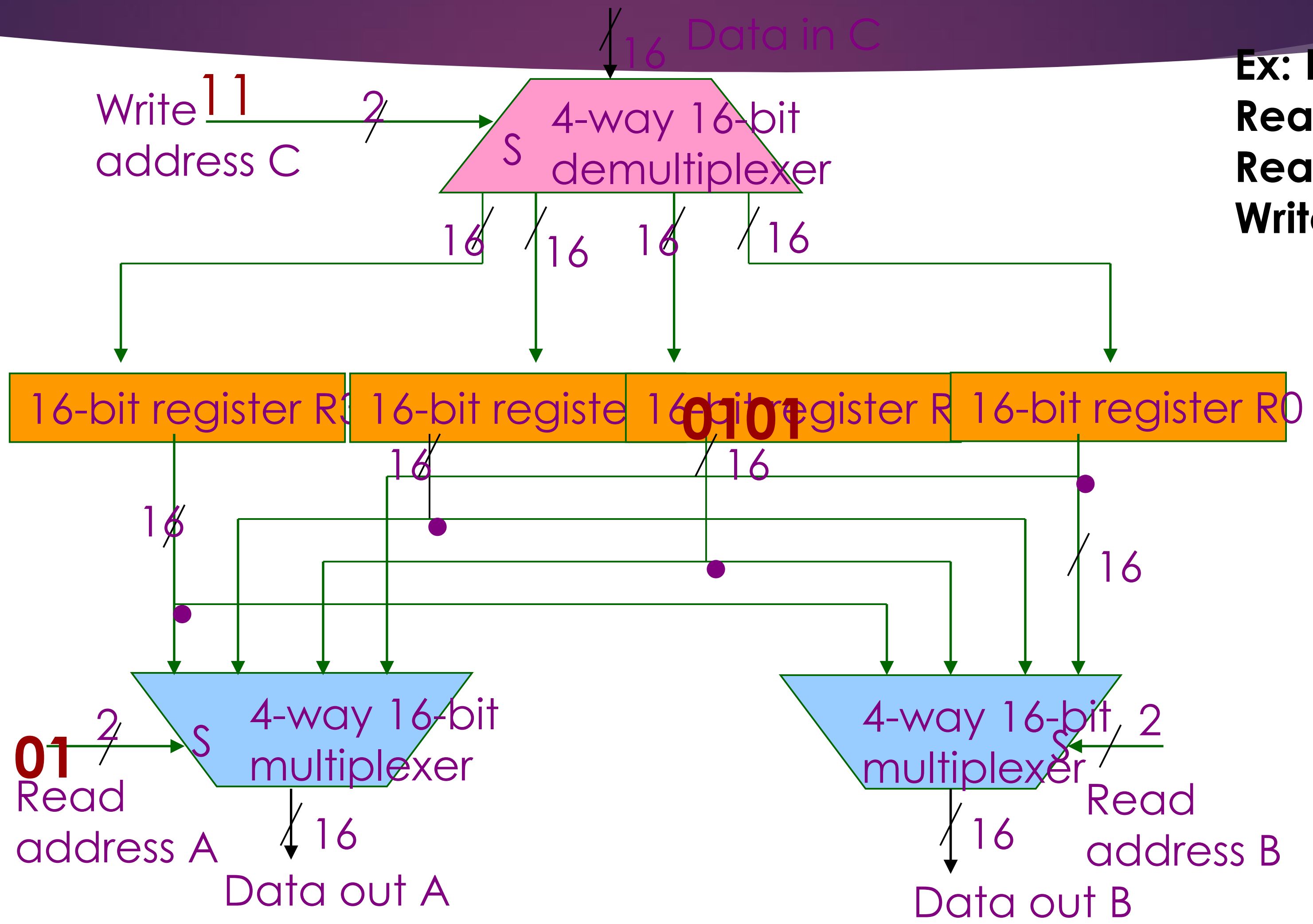
Example Register Organizations



A register file with three access ports - symbol

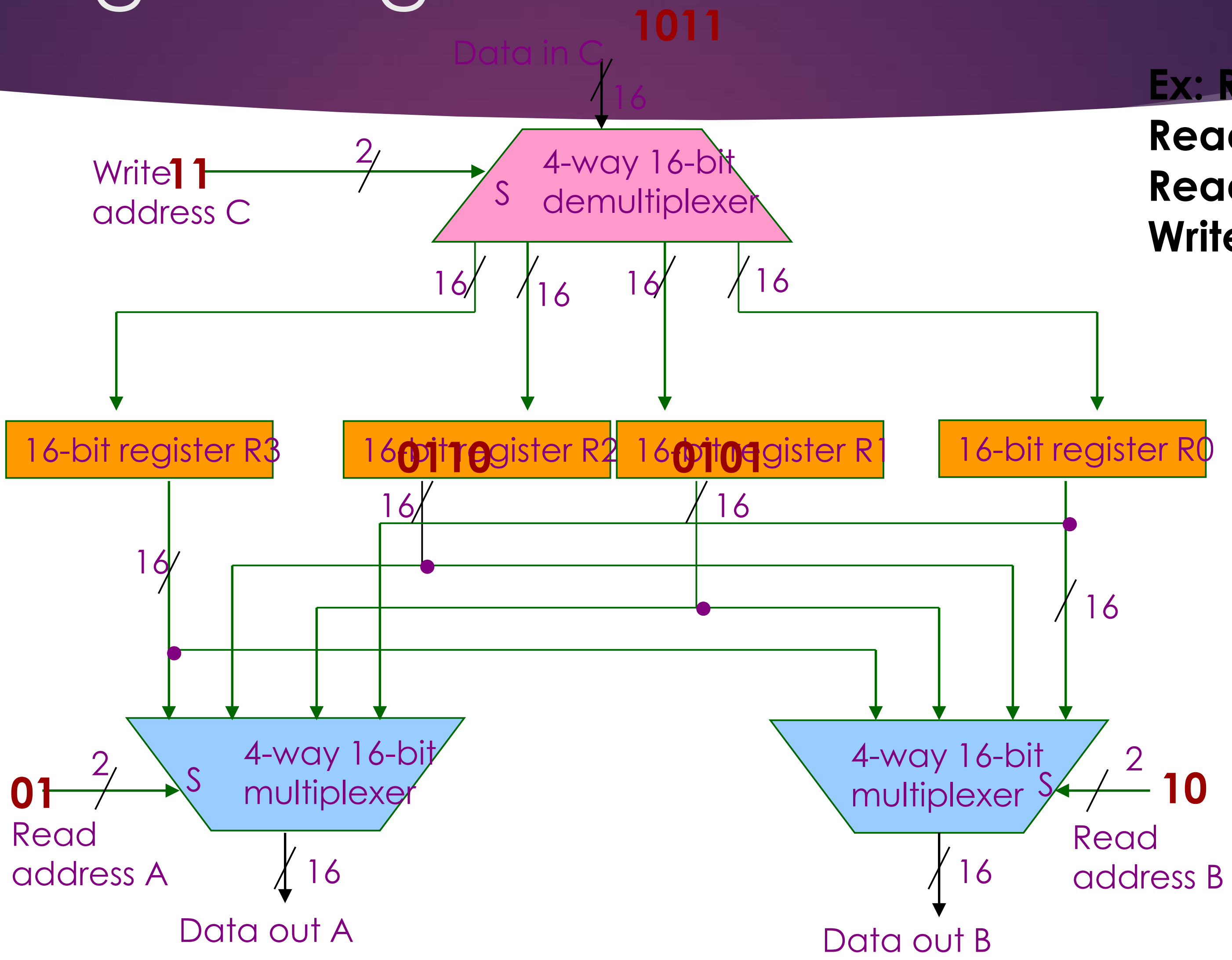


A Register File with three access ports – logic diagram



Ex: R3 ← R1 + R2
Read Address A = 01
Read Address B = 10
Write Address C = 11

A Register File with three access ports – logic diagram



Ex: R3 ← R1 + R2
Read Address A = 01
Read Address B = 10
Write Address C = 11

Questions

- If the 8 registers are used
  - How many bits are needed for read/write address?
  - What is the size of the demultiplexer and multiplexer required?
  - How many multiplexers and demultiplexer's are required to perform three read and one write operation?
- If 4 multiplexers are used, how many parallel reads can be performed?
- If 2 demultiplexers are used, how many parallel writes can be performed?
- Design a register file that stores eight 32 bit numbers and has one read and one write port.